# 第 11 节 实操任务 3：实现 Planner Agent...

> 学习目标：Planner 节点作为图入口，根据 `PLANNER_TARGET_COUNT` 环境变量输出三档策略（lite/standard/full）
> 目标文件：`workflows/planner.py`（新增）+ 修改 `workflows/graph.py`

---

## 背景

实操 2 把 6 节点工作流跑通了。但有两处硬编码：

- `route_after_review` 里写死 `iteration >= 3`
- `collector.py` / `organizer.py` / `reviewer.py` 里的默认值

生产系统要能根据输入规模调整策略——数据多时要保守，数据少时要激进。这就是 **Planner Agent** 的职责：**只规划不执行（Plan, don't execute）**。

Planner 挂在图的最前面，输出一个 `plan` dict 写入 state，下游节点读取它调整行为。

---

## 步骤 1：创建 planner.py

**提示词：**

```
请帮我编写 workflows/planner.py：

需求：
1. plan_strategy(target_count=None) 函数，根据目标采集量返回策略 dict
2. 三档策略：
   - lite (target<10): per_source_limit=5, relevance_threshold=0.7, max_
   - standard (10<=target<20): 10, 0.5, 2
   - full (target>=20): 20, 0.4, 3
3. target_count 默认从环境变量 PLANNER_TARGET_COUNT 读取（默认 10）
4. planner_node(state) 函数：LangGraph 节点包装，调 plan_strategy 并返回 {"p
5. 每个策略 dict 包含 rationale 字段说明"为什么这么选"
```

**参考实现：** `workflows/planner.py`

```
"""Planner Agent — 动态规划节点（V3 流水线节点 ①）

核心原则：只规划不执行（Plan, don't execute）。
Planner 的输出写入 state["plan"]，被下游 Collector/Organizer/Reviewer 共同消
"""

import os
```

```


def plan_strategy(target_count: int | None = None) -> dict:
    """根据目标采集量选择策略 — 最小可运行 Planner"""
    if target_count is None:
        target_count = int(os.getenv("PLANNER_TARGET_COUNT", "10"))

    if target_count >= 20:
        return {
            "strategy": "full",
            "per_source_limit": 20,
            "relevance_threshold": 0.4,
            "max_iterations": 3,
            "rationale": f"目标 {target_count} 条，启用深度模式（质量优先）",
        }
    elif target_count >= 10:
        return {
            "strategy": "standard",
            "per_source_limit": 10,
            "relevance_threshold": 0.5,
            "max_iterations": 2,
            "rationale": f"目标 {target_count} 条，启用标准模式（平衡）",
        }
    else:
        return {
            "strategy": "lite",
            "per_source_limit": 5,
            "relevance_threshold": 0.7,
            "max_iterations": 1,
            "rationale": f"目标 {target_count} 条，启用精简模式（成本优先）",
        }


def planner_node(state: dict) -> dict:
    """LangGraph 节点：把策略写入 state["plan"]"""
    plan = plan_strategy()
    print(
        f"[Planner] 策略={plan['strategy']}, 每源={plan['per_source_limit
        f"阈值={plan['relevance_threshold']}, {plan['rationale']}"
    )
    return {"plan": plan}
```

---

## 步骤 2：修改 graph.py 把 plan 挂到入口

```
"""LangGraph 工作流图 — 第 11 节完整 7 节点版"""

```

```python
from langgraph.graph import END, StateGraph

from workflows.analyzer import analyze_node
from workflows.collector import collect_node
from workflows.human_flag import human_flag_node
from workflows.organizer import organize_node
from workflows.planner import planner_node   # ← 新增
from workflows.reviewer import review_node
from workflows.reviser import revise_node
from workflows.state import KBState


def route_after_review(state: KBState) -> str:
    """条件路由：读 state["plan"]["max_iterations"]，不再硬编码 3"""
    plan = state.get("plan", {}) or {}
    max_iter = int(plan.get("max_iterations", 3))
    iteration = state.get("iteration", 0)

    if state.get("review_passed", False):
        return "organize"
    elif iteration >= max_iter:
        return "human_flag"
    else:
        return "revise"


def build_graph() -> StateGraph:
    graph = StateGraph(KBState)

    # 【新增】注册 plan 节点
    graph.add_node("plan", planner_node)
    graph.add_node("collect", collect_node)
    graph.add_node("analyze", analyze_node)
    graph.add_node("review", review_node)
    graph.add_node("revise", revise_node)
    graph.add_node("organize", organize_node)
    graph.add_node("human_flag", human_flag_node)

    # 【新增】plan → collect 边
    graph.add_edge("plan", "collect")
    graph.add_edge("collect", "analyze")
    graph.add_edge("analyze", "review")

    graph.add_conditional_edges(
        "review",
        route_after_review,
        {
            "organize": "organize",
```

```
            "revise": "revise",
            "human_flag": "human_flag",
        },
    )

    graph.add_edge("revise", "review")
    graph.add_edge("organize", END)
    graph.add_edge("human_flag", END)

    # 【修改】入口从 "collect" 改为 "plan"
    graph.set_entry_point("plan")
    return graph


app = build_graph().compile()
```

注意两处关键变化：

1. `set_entry_point("plan")` — 入口改为 plan
2. `route_after_review` 里 `max_iter` 从 `state["plan"]` 读取，不再硬编码

---

## 步骤 3：验证 collector / organizer / reviewer 已读 plan 字段

第 10 节写的 `collector.py` 和 `organizer.py` 已经用过 `state.get("plan", {}).get("per_source_limit", 10)` 这种模式。检查一下它们：

```
grep -n 'plan.*per_source_limit\|plan.*relevance_threshold' workflows/co
```

应该看到：

- `collector.py` 读 `per_source_limit`
- `organizer.py` 读 `relevance_threshold`

如果没读，补上即可。

Reviewer 需要加一行读 `max_iterations`（虽然当前的 `route_after_review` 已经用了它，Reviewer 内部也可以兜底）——但这不是必须的。

---

## 步骤 4：运行验证

### 4.1 默认（standard 策略）

```
cd ~/ai-knowledge-base/v3-multi-agent
python3 -m workflows.graph
```

**期望输出：**

```
[Planner] 策略=standard, 每源=10 条, 阈值=0.5, 目标 10 条, 启用标准模式（平衡）
[Collector] 采集到 10 条原始数据
...
```

### 4.2 切换到 lite

```
PLANNER_TARGET_COUNT=5 python3 -m workflows.graph
```

**期望输出：**

```
[Planner] 策略=lite, 每源=5 条, 阈值=0.7, 目标 5 条, 启用精简模式（成本优先）
[Collector] 采集到 5 条原始数据
[Reviewer] 加权总分: X/10, 通过: Y (第 1 次审核)
...
```

注意 lite 策略下 `max_iterations=1` ——如果首次就不通过，直接走 `human_flag`。

### 4.3 切换到 full

```
PLANNER_TARGET_COUNT=30 python3 -m workflows.graph
```

**期望输出：**

```
[Planner] 策略=full, 每源=20 条, 阈值=0.4, 目标 30 条, 启用深度模式（质量优先）
[Collector] 采集到 20 条原始数据
...
```

---

## 步骤 5：验证清单

| 检查项 | 期望 | 实际 |
|---|---|---|
| workflows/planner.py 存在 | 是 | |
| plan_strategy(5) 返回 lite | 是 | |
| plan_strategy(15) 返回 standard | 是 | |
| plan_strategy(30) 返回 full | 是 | |
| graph 的 entry_point 是 plan | 是 | |
| route_after_review 读 plan.max_iterations | 是 | |
| 三种策略都能跑通 | 是 | |

---

## 步骤 6：提交到 Git

```
git add workflows/planner.py workflows/graph.py
git commit -m "feat: add Planner agent as graph entry with dynamic strat
```

---

## 7 节点最终拓扑

```
① plan → ② collect → ③ analyze → ④ review ┬─[pass]──────→ ⑥ organize → E
                                           │
                                           ├─[fail<max]──→ ⑤ revise → ④
                                           │
                                           └─[>=max]──────→ ⑦ human_flag -
```

| 节点 | 文件 | 职责 |
| --- | --- | --- |
| ① Planner | `workflows/planner.py` | 动态规划策略 |
| ② Collector | `workflows/collector.py` | 数据采集 |
| ③ Analyzer | `workflows/analyzer.py` | LLM 单条分析 |
| ④ Reviewer | `workflows/reviewer.py` | 5 维加权审核 |
| ⑤ Reviser | `workflows/reviser.py` | 读反馈定向修改 |
| ⑥ Organizer | `workflows/organizer.py` | 整理入库（正常终点） |
| ⑦ HumanFlag | `workflows/human_flag.py` | 人工介入（异常终点） |

**一个Agent对应一文件，文件名 = Agent 名，一目了然。**

**完成！** V3 的 7 节点工作流全部搭建完毕，Planner / Reviewer / Reviser / HumanFlag 四个“质量闭环 Agent”全部到位。

下节课（第 12 节）我们继续给系统加成本控制、安全防护、评估测试，让我们的 Agent 从“能跑”升级到“能上生产”。